

Technical Design Document

„Debugger3D“

Modulnummer: 4FSC0PD003/4 0326

Modulname: Games Programming

Abschluss: Games Programming Diploma

Semester: März 2026

Name: Bartosz Chelminiak

Campus: Köln

Land: Deutschland

Selbstständigkeitserklärung

Hiermit bestätige ich, dass ich die vorliegende Arbeit eigenständig erarbeitet habe und alle Hilfsmittel sowie Inhalte von Dritten vollständig angegeben wurden. Hierunter fällt zum Beispiel die korrekte Belegarbeit für die direkte oder sinngemäße Übernahme von Texten, Bild-, Audio- und Videomaterial sowie Code etc. Weiterhin die klare Kennzeichnung von Inhalten, die von anderen Personen oder technischen Hilfsmitteln wie z. B. einer künstlichen Intelligenz erstellt wurden.

Düren, den 22.08.2026

Bartosz Chelminiak

Ort, Datum

Unterschrift Student/in

Rechtevereinbarung

Hiermit räume ich dem SAE Institut das nicht exklusive, jedoch zeitlich und örtlich unbeschränkte Recht ein, die vorliegende Arbeit zum Zweck der Ausbildung sowie der Darstellung von Ausbildungsinhalten zu speichern und für Personen des SAE Instituts zugänglich zu machen.

Düren, den 22.08.2026

Bartosz Chelminiak

Ort, Datum

Unterschrift Student/in

Inhaltsverzeichnis

1.Einleitung.....	3
2.Inhalt und Features.....	3
3.UML Klassendiagramm.....	4
4.Programmablaufplan.....	5
5.Aufbau.....	6
6.Skalierbarkeit.....	7
7.Threadoptimierung.....	7-9
8.Engine Tool.....	10
9.Asset Integration.....	11

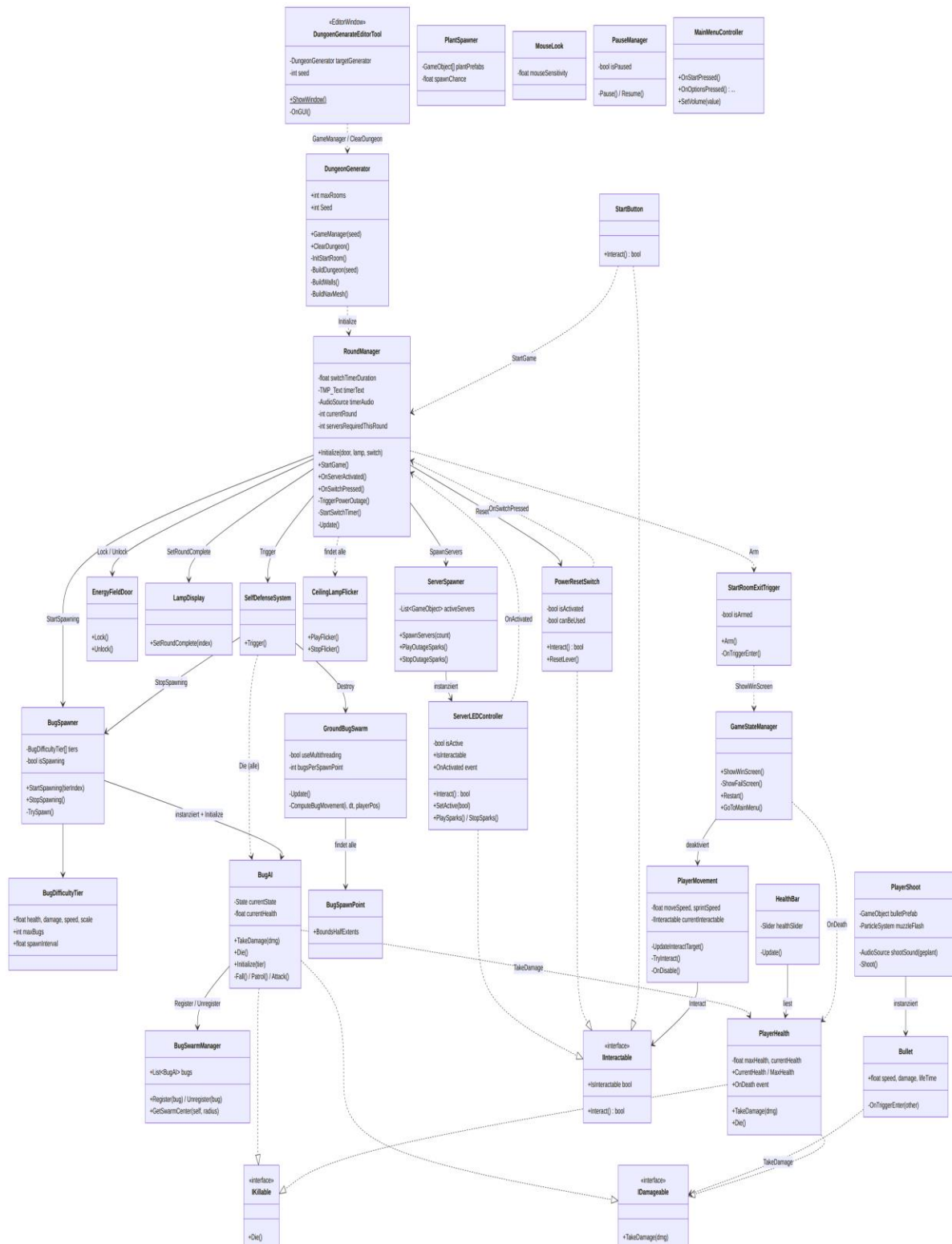
1 Einleitung

Dieses Dokument ist der Software-Entwurfsplan für Debugger3D. Es zeigt, aus welchen Klassen das Spiel besteht, wie sie zusammenhängen und in welcher Reihenfolge sie zur Laufzeit aufgerufen werden. Dazu kommen Threadoptimierung, Editor-Tool und die Lizenzanalyse der Assets.

2 Inhalt und Features

- Prozeduraler Dungeon aus elf Raum-Prefabs, per Seed wiederholbar, NavMesh wird zur Laufzeit gebacken
- Handgebauter Startraum mit Startknopf, Energiefeld-Tür, Lampenanzeige, Schalter und Sieg-Trigger
- Drei Runden mit 3, 6 und 9 Servern; nach jeder Aktivierung aller Server ein Stromausfall, der Schalter beendet die Runde
- Spinnen-KI mit Zuständen Fall, Patrol, Chase, Attack und einer Schwierigkeitsstufe pro Runde
- Pistole mit Projektil, Lebenspunkte, Game Over bei null
- Runde 3: Schalter 60 Sekunden gesperrt, danach Selbstverteidigungssystem, das alle Gegner entfernt
- Boden-Käfer mit Schwarmverhalten, auf mehreren CPU-Kernen berechnet
- Interaktionssystem über Raycast und ein gemeinsames Interface
- Hauptmenü, HUD (Lebensbalken, Interact-Hinweis, Timer), Pause-, Win- und Fail-Screen
- Editor-Werkzeug zum Aufbauen des Dungeons ohne Play-Modus
- Audio: Menümusik, Ambience, Schuss, zwei Ansagen

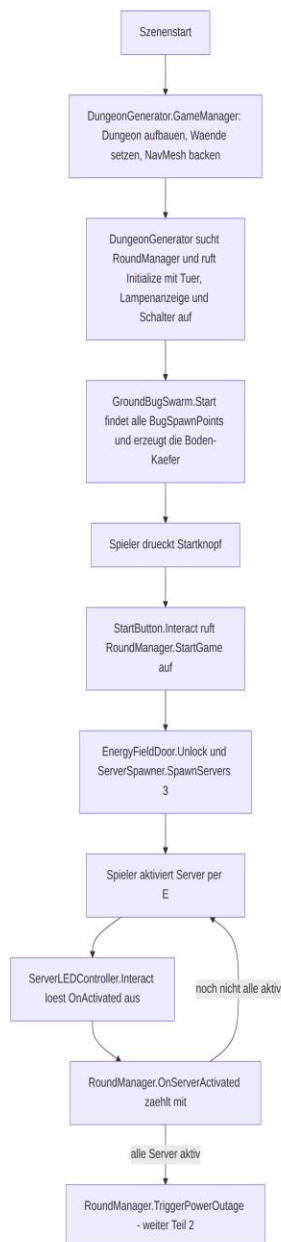
3 Klassendiagramm



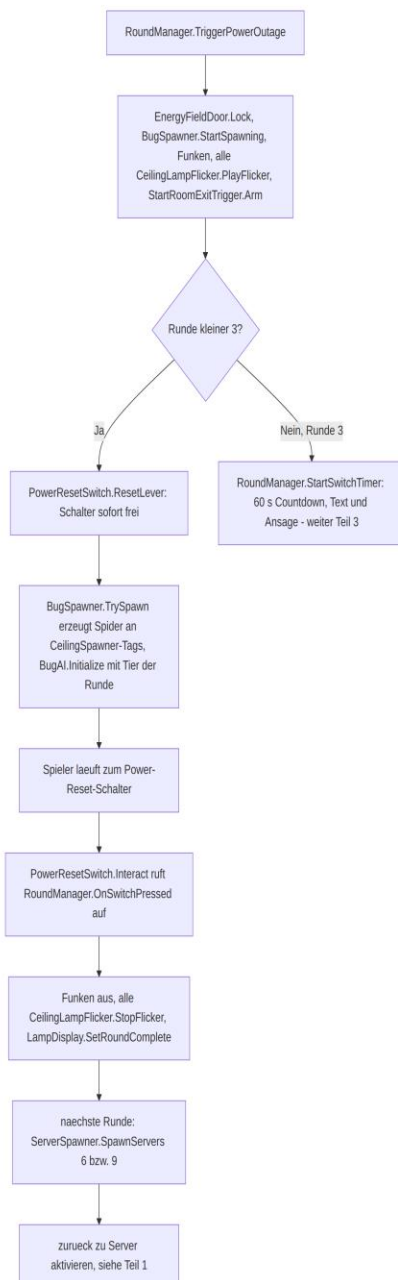
4 Programmablaufplan

Die folgenden Abbildungen zeigen den aktuellen Programmablauf: die Erzeugung der Objekte beim Szenenstart und die Aufrufe der Klassen untereinander während der drei Runden. Der Ablauf ist in drei Teile geteilt, weil er sich nach dem ersten Stromausfall wiederholt.

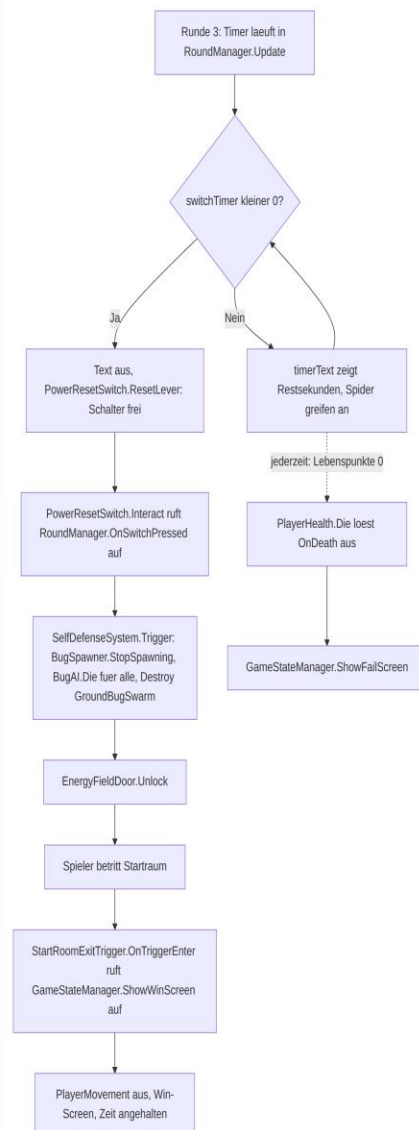
Teil 1: Aufbau und Serveraktivierung



Teil 2: Stromausfall und Rundenschleife



Teil 3: Runde 3, Timer und Spielende



5 Aufbau

- **Interfaces:** PlayerMovement ruft nur `IIteractable.Interact()` auf und muss nicht wissen, ob es ein Server, Knopf oder Schalter ist. Kugeln treffen Ziele nur über `IDamageable`.
- **Events:** Server und PlayerHealth lösen Events aus, statt RoundManager oder UI direkt zu kennen.
- **Spawnpunkte in den Prefabs:** Server, Spinnen, Käfer und Pflanzen finden ihre Spawnpunkte selbst über Tags. Der Generator stellt nur Räume auf.

6 Skalierbarkeit

Eine weitere Runde ist ein Eintrag mehr im Array der Schwierigkeitsstufen. Ein neuer Raumtyp braucht nur den passenden Connector. Neue interagierbare Objekte funktionieren über `IIteractable` sofort. Die Raumanzahl ist ein Inspector-Wert; die Grenze setzt die Performance, die durch die Threadoptimierung deutlich höher liegt.

7 Threadoptimierung

Ausgelagert habe ich die Bewegung der Boden-Käfer. Die Käfer sind Boids, also kleine Objekte, die sich an ihren Nachbarn orientieren: Abstand halten und in eine ähnliche Richtung laufen. Dafür muss jeder Käfer in jedem Frame alle anderen Käfer in seinem Raum "anschauen". Bei ein paar tausend Käfern im Dungeon ist das die aufwendigste Rechnung im ganzen Spiel. Ich habe die Käfer extra für diese Aufgabe gebaut, weil jeder Käfer für sich allein rechnet und man die Arbeit deshalb gut auf mehrere Kerne verteilen kann.

Die Rechnung für einen Käfer steckt in einer eigenen Methode. Im Update rufe ich sie für alle Käfer auf, entweder ganz normal nacheinander in einer for-Schleife, oder mit `Parallel.For`, eine Schleife die die Käfer automatisch auf alle Prozessorkerne verteilt. Über einen Haken im Inspector kann ich zwischen beiden Varianten umschalten und vergleichen.

Bild 30 Bugs VOR Threadoptimierung

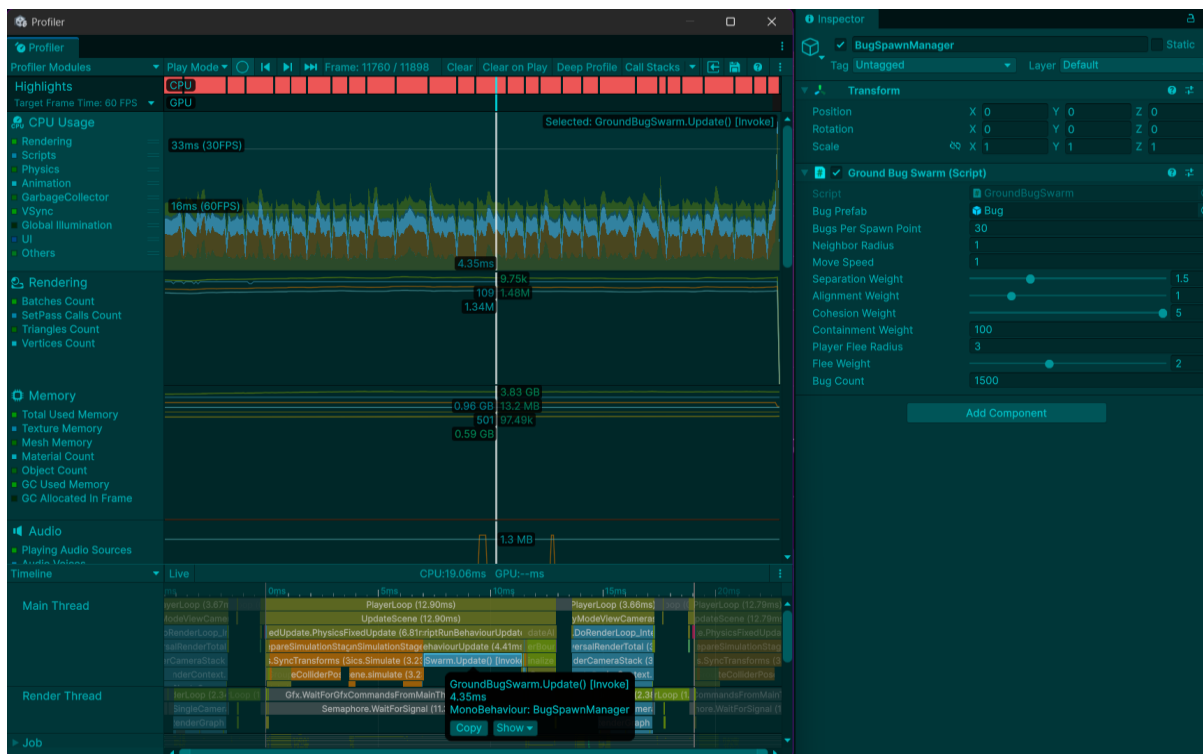


Bild mit 30 Bugs NACH Threadoptimierung

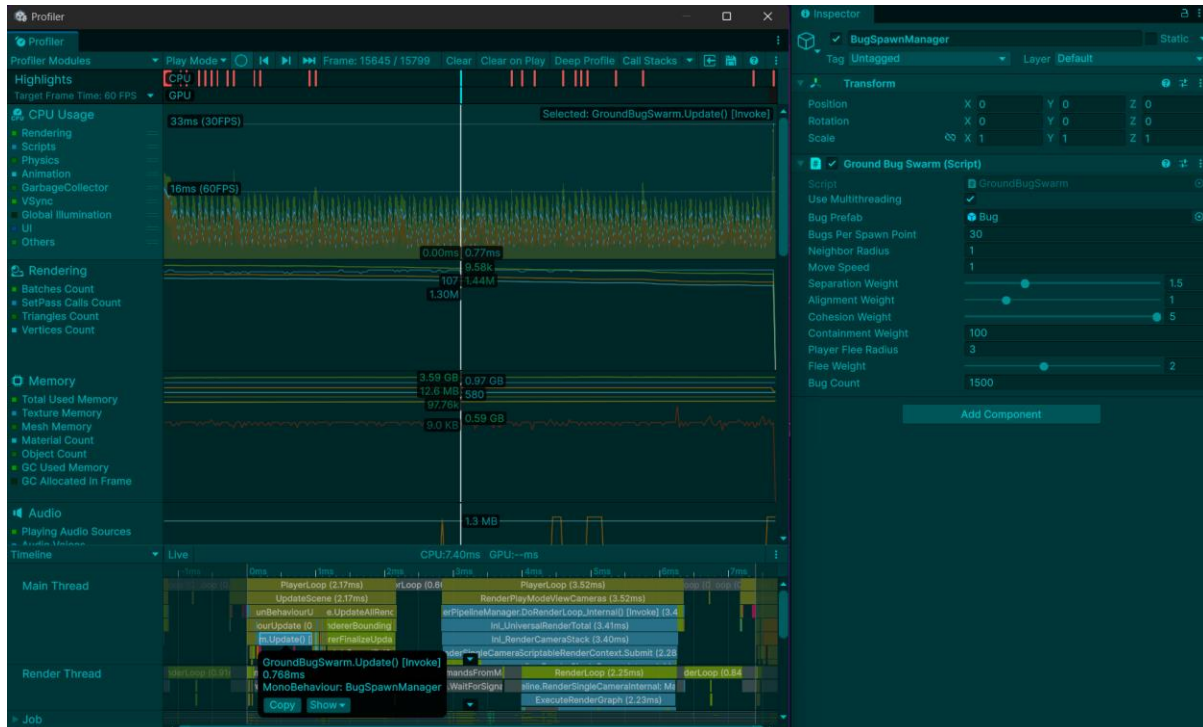


Bild mit 60 Bugs VOR Threadoptimierung

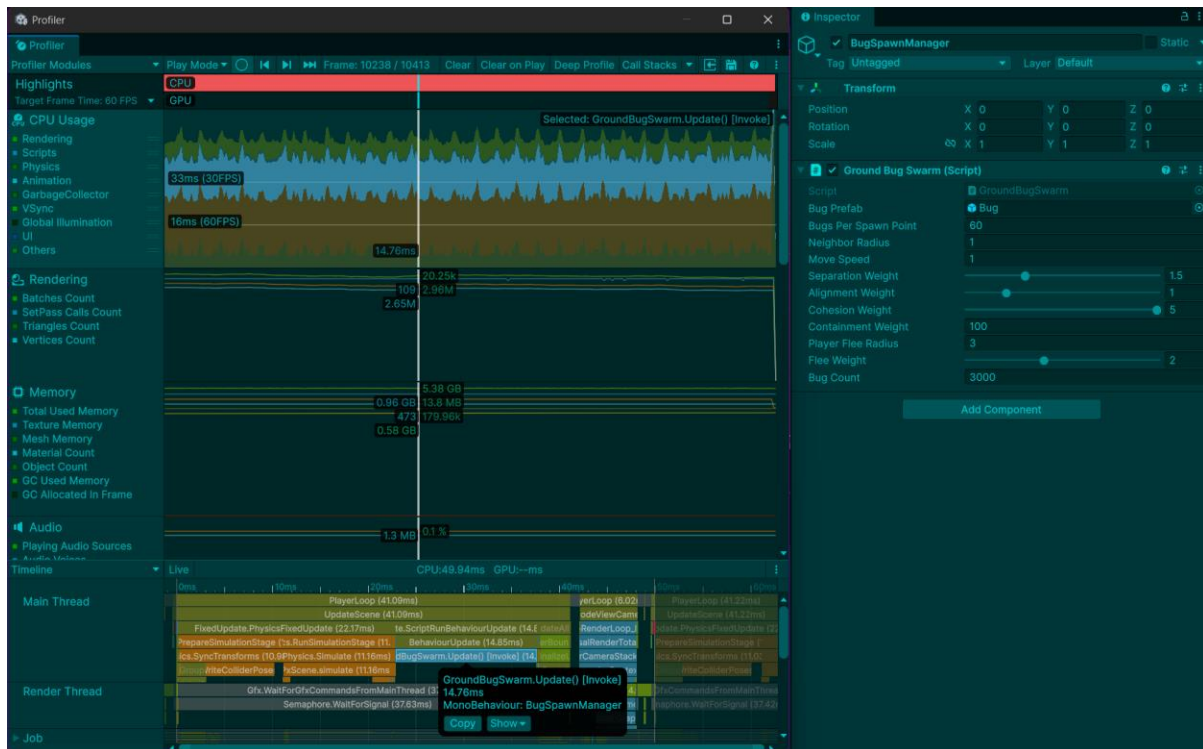
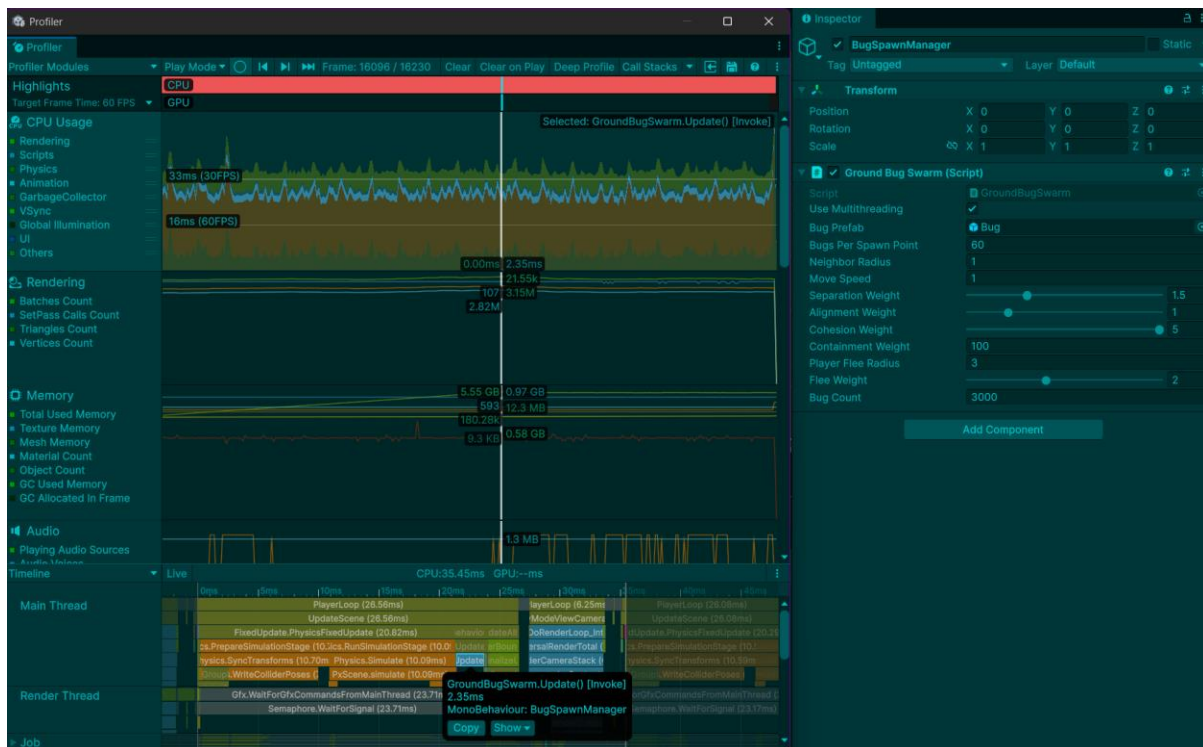


Bild mit 60 Bugs NACH Threadoptimierung



Gemessen habe ich mit dem Unity Profiler. Der zeigt an, wie viele Millisekunden jede Methode pro Frame braucht. Getestet habe ich mit 50 Räumen auf meinem Desktop: AMD Ryzen 7 5800X3D (8 Kerne), AMD Radeon RX 7900 XT, 32 GB RAM, Windows 11, Unity 6.

Ergebnisse:

- 1500 Käfer (30 pro Raum): vorher 4,35 ms, nachher 0,77 ms, also 5,7× schneller
- 3000 Käfer (60 pro Raum): vorher 14,75 ms, nachher 2,35 ms, also 6,3× schneller

Das Spiel läuft mit Multithreading etwa sechsmal schneller. Mein Prozessor hat acht Kerne, also könnte man theoretisch achtmal erwarten. Die Arbeit auf die Kerne zu verteilen und die Ergebnisse wieder einzusammeln kostet selbst auch ein bisschen Zeit, darum ist weniger als 8x logisch. Trotzdem zeigt der Wert, dass die Berechnung wirklich parallel läuft.

Bei der Arbeit daran ist mir im Profiler aufgefallen, dass die Physik viel Leistung gefressen hat, obwohl die Käfer gar keine Physik benutzen. Der Grund: Ich hatte das Käfer-Prefab selbst in Unity aus 3D-Objects erstellt und dabei vergessen die Collider zu entfernen. Das Käferprefab hatte damit also noch elf Collider aus dem 3D-Modell dran. Nachdem ich zehn davon gelöscht habe, lief das Spiel deutlich flüssiger. Der Profiler hat sich somit also direkt als praktisches Tool zum Auffinden von weiteren Fehlern bewiesen.

8 Engine-Tool

Das Tool ist ein Editor Fenster unter Tools → Dungeon Generator mit Feldern für DungeonGenerator, Seed und Raumanzahl und den Knöpfen Seed würfeln, Generate und Delete. Es hat keine eigene Logik, sondern ruft nur GameManager und ClearDungeon des Generators auf. Damit lässt sich der Dungeon in der Szene aufbauen und prüfen, ohne das Spiel zu starten – beim Bauen des Dungeons die wichtigste Hilfe.

9 Asset-Integration

Vier Asset-Pakete fremder Autoren mit drei verschiedenen Lizenztypen:

- **Spider – Low Poly – Animated** (RetroVandal, itch.io), **CC BY 4.0**. Kommerziell und privat erlaubt, Bearbeitung erlaubt, keine Gewährleistung, kein Copyleft. Einzige Pflicht: Namensnennung – steht im Credits-Panel des Hauptmenüs.
- **Free Sci-Fi Office Pack** (Terresquall) und **Gogo Casual Free Plants Pack**, beide **Unity Asset Store EULA**. Kommerzielle Nutzung im fertigen Spiel erlaubt, aber die Rohdaten dürfen nicht einzeln weitergegeben oder weiterverkauft werden. Keine Namensnennung nötig, keine Gewährleistung, kein Copyleft.
- **Retro Weapon Pack, Readme-Freigabe** („Free for personal and commercial projects. No attribution required.“). Keine formale Lizenz, sondern eine Erlaubnis des Autors, ohne ausführlichen Lizenztext.

Alle vier Lizenzen sind miteinander verträglich: Keine hat Copyleft, keine verbietet kommerzielle Nutzung. Die wichtigste Erkenntnis für mich: „Kostenlos“ heißt nicht „ohne Bedingungen“ – ohne den Credits-Eintrag würde ich beim Spinnenmodell die Lizenz verletzen. Trotzdem sind die anderen Assets ebenfalls in den Credits aus Achtung vor der Schöpfung.